

Figure 1: Schematic overview of the compiler passes that are considered in this paper.

This paper is organized as follows. Section 2 describes the design of our parallel compiler and Section 3 its implementation on modern GPUs. Section 4 presents an evaluation of the compiler’s parallel performance. Section 5 describes future work. Section 6 discusses related work in parallel compilation. Section 7 concludes the paper.

2 DESIGN

In this paper, we consider the compilation passes from parsing of the source code of a program until the generation of machine code, summarized in Figure 1. To parallelize a compiler there are two approaches: run many, possibly different, passes in parallel, or parallelize each pass individually. Given the SIMD nature of GPU architectures, the latter is the more suitable choice, and we describe the individual parallelization of each pass. Another important property of GPU architecture is the reliance on streaming memory access and that codes involving frequent branching and irregular memory access (e.g., due to indirection arrays or pointer-linked data structures) are ill suited. In fact, traditional compilers are for the most part centered around the manipulation of pointer-linked trees using recursive functions that involve heavy branching. Therefore, in our design an inverted tree data structure is used instead of pointer-linked trees. The inverted tree is array-based and can be manipulated in a straightforward manner using parallel operations.

Figure 2 illustrates the differences between these. The inverted tree was recently proposed for the implementation of compiler transformations on the GPU by Hsu, et al. [10]. We use a variant of the inverted tree and store the parse tree in pre-order and Abstract Syntax Tree (AST) in post-order. Additionally, we use different algorithms to manipulate the inverted tree. Whereas pointer-linked trees are manipulated using recursive operations, we use iterative, loop-based array accesses which are easier to parallelize. The basic inverted tree data structure is augmented with additional fields and node types as the compilation progresses.

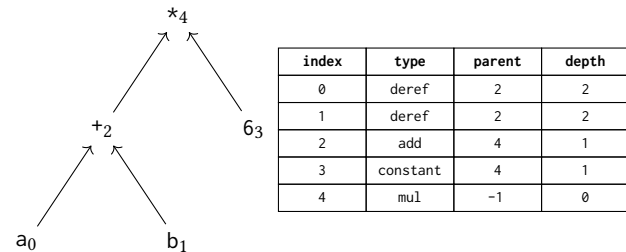


Figure 2: Left: illustration of regular tree for expression $(a+b) * 6$, subscript indicates node index. Right: the corresponding inverted tree data structure in post-order.

The parallel operations are implementing using parallel array primitives. This allows for a reduced development time compared to writing the various GPU kernels by hand and results in a portable code base as efficient implementations of these primitives exist for different architectures. A pure functional programming language, in our case Futhark [6], is used to compose these primitives into programs. In such languages no expression may have side effects and this makes it easier to reason about optimizing high-level constructs into parallel primitives. Many of the primitives that are used in the implementation, such as parallel map, reduction, scan and prefix sum were first described by Hillis and Steele [9].

2.1 Lexical Analysis

The first stage performs *lexical analysis* according to a *lexical grammar* to partition the input into a sequence of *tokens*. The main problem of parallel lexical analysis presents itself when grammars with more complicated tokens are introduced, such as comments and quoted strings. In such a case processors that are parsing sections of the input in parallel may need to look back to the start of the input to determine the current state, which is detrimental to the performance. We address this by representing the grammar as a single deterministic finite automaton that is run once for each token in the input. Subsequently, we employ the algorithm proposed by Hillis & Steele [9] to evaluate this automaton on all characters of the input string in parallel. This can be implemented in terms of lookup tables generated ahead of the lexical analysis, see Section 3.1.

2.2 Parsing

In the *parsing* stage the syntactic structure of the input is validated according to the language’s syntactic grammar and a *parse tree* is built. This is usually performed using a *push-down automaton*, which requires the current state of the stack to be known to parse a section of the input. This causes subsequent sections to be dependent on each other, refraining these from being parsed in parallel. We break this dependence chain by making it possible to determine the top of the stack from the *previous* few input symbols, relative to the current input position, instead of maintaining an actual stack. Sections of the input are now processed in parallel by deriving the initial contents of the stack from preceding input symbols, parsing each section using a regular predictive *LL* parser and recording the push and pop operations. The recorded operations (*stack configuration changes*) are concatenated to validate the input and to produce the final parse. From the final parse the parse tree is constructed.

The foundation of a parallel parsing algorithm that takes this approach, and that we have implemented, is described in [20]. This algorithm is capable of deterministically parsing languages of the newly defined *LLP*(q, k) grammar class in $O(\log n)$ parallel time, where n is the number of tokens to parse. Within this definition,